

Performance Evaluation of Computer Systems

Class 09 - Simulator

Pedro Braconnot Velloso

Agenda

- Random number generation
 - Using C++, Python or Java
- Building our own simulator
 - Object oriented simulator
- Simulating a simple system

The discrete event simulator

Discrete event simulator

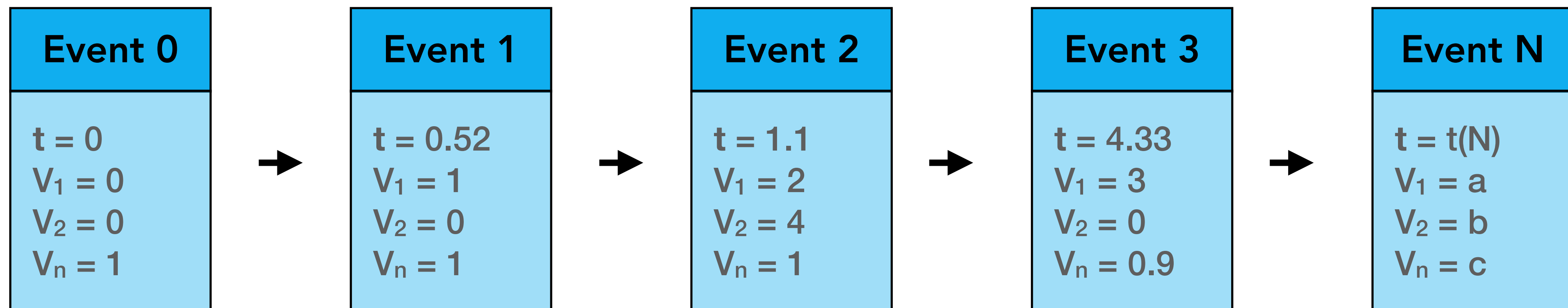
- Simulation a probabilistic model involves
 - Generating the stochastic mechanisms of the model
 - Observe the resulting flow of the model over time
 - Compute the quantities of interest
- Example
 - A client sending a request to web server

Simulation via discrete events

- The key elements are variable and events
- We will keep track of certain variables
 - The amount of time it has elapsed since the beginning of the simulations
 - Counter variables
 - System state variables
- Whenever an event occurs
 - These variables may change or be updated

Events

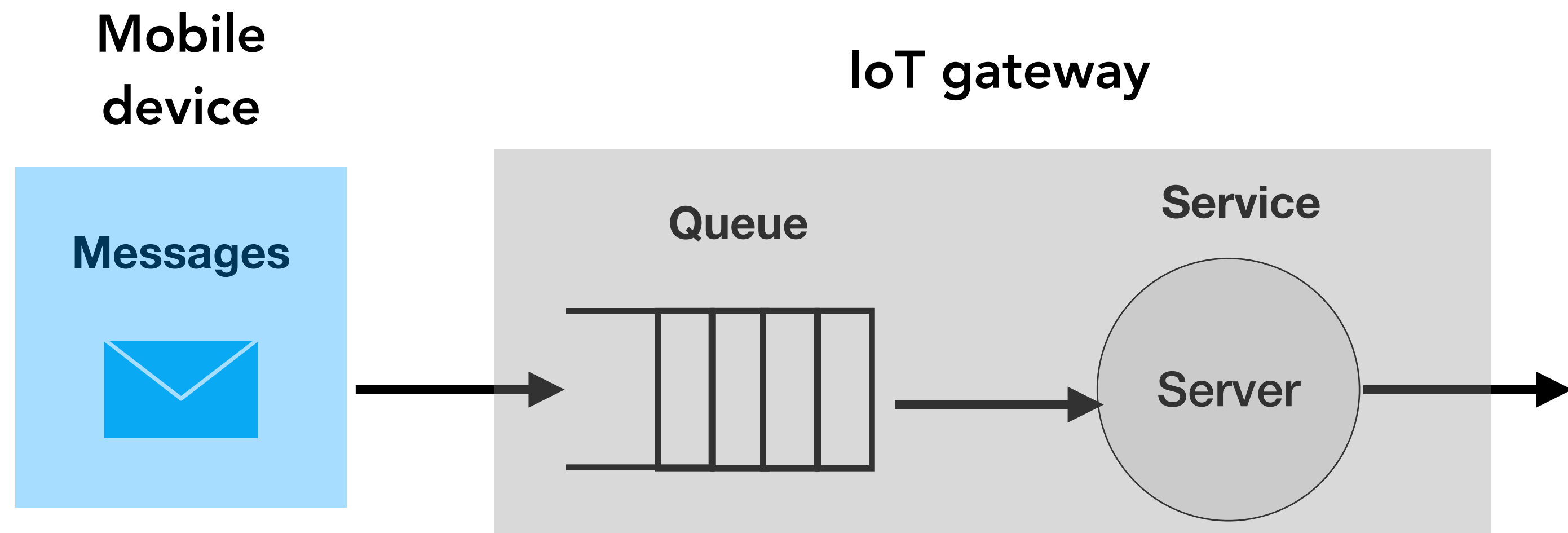
- Events are kept in a ordered list
- They are organised by the scheduler
 - Current time t $\rightarrow$ always the time of the current event



Building a simulator

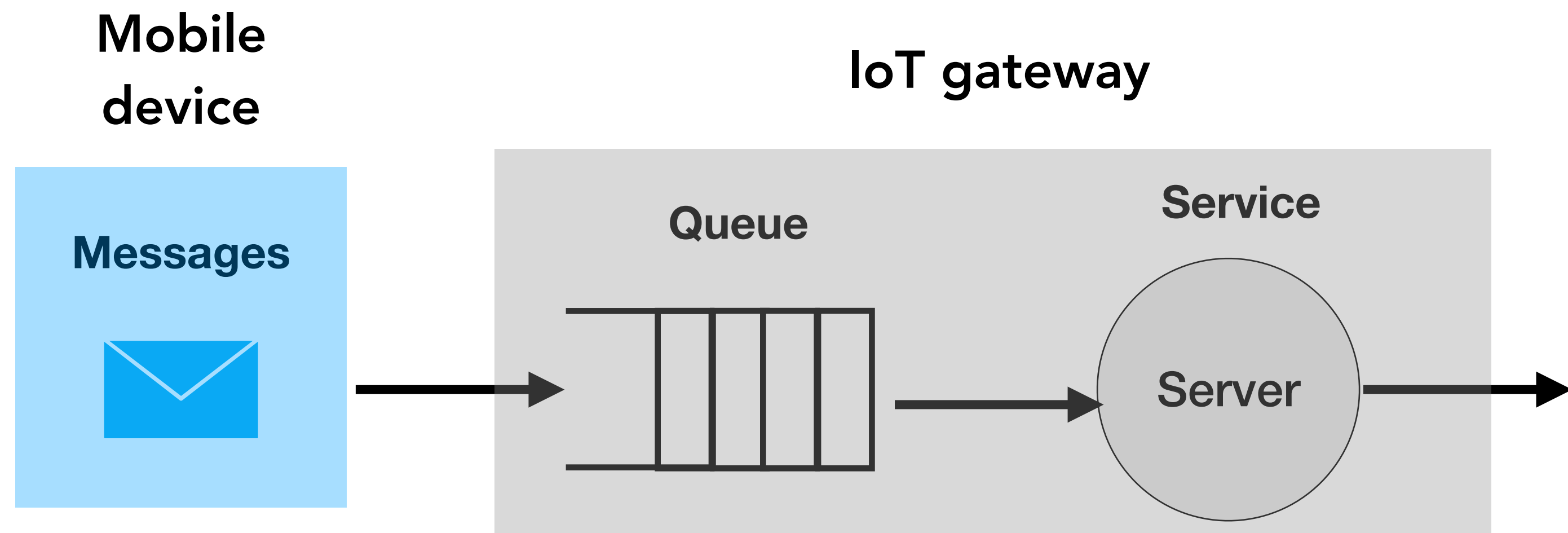
Main components

- Engine (Main)
- Scheduler
- Client
- Gateway
- Server
- Queue



Main components

- Each component is a class
- One file for each class
- Auxiliary classes
 - Event
 - Message



Basic events

- Type
 - SEND_MSG
 - The client sends a message to the gateway
 - When a message is sent
 - It takes 1 second to arrive at the gateway
 - RECV_MSG
 - The gateway receives de message
 - MSG_DEPT
 - The gateway completes a service

Basic events

- Message
- Timestamp
- Source
- Destination

Basic events

- Message are created with
 - Inter-arrival time exponentially distributed
 - With mean λ
- The service time of the gateway
 - When a message leaves the systems
 - The service is completed
 - Service time exponentially distributed
 - With mean μ

Basic metrics

- The instant number
 - Requests in the system and in the queue
- The total number of requests
 - Requests in the system and in the queue
- The average number requests
 - Requests in the system and in the queue
- The same for the waiting time

Tests for each class implemented

- Print method
 - Print all the class members
 - Print_Message()
 - Print_Event()
 - Etc.
- One test function in the in the Engine file per class
 - Test_Message()
 - Test_Event()
 - Etc.

Engine Class

- Contains the Main() function
- Store the main parameters
 - Number of clients
 - Number of servers in the gateway
 - Simulation time
- We have just one gateway
- Methods
 - GenerateTrace(Event e)
 - CreateClients(int nClients)
 - Run()
 - Etc.

Trace

- After each event
 - Generate the trace

time	node	event	source	dest.	msgID
1.202	1	SEND	1	0	1
1.916	0	RECV	1	0	1
2.320	3	SEND	3	0	2
2.391	0	RECV	3	0	2
4.572	0	DEPT	1	0	1
5.916	0	DEPT	3	0	2

Event

- Members
 - EventID
 - Message
 - EventType
- Methods
 - Constructor and destructor
 - Event(), ~Event()
 - Setters and Getters
 - SetEventTime()
 - GetEventTime()
 - SetEventType()
 - GetEventType()

Message

- messageId
- source
- destination
- Constructor
- Getters and setters

Scheduler

- Events are stored in chronological order
- GetEvent()
- GetCurrentTime()
- AddEvent()
 - Events must be inserted in the correct position
 - Cannot use sort() every time

Evaluation

- Implementation in class
 - 100%
- Implementation at home
 - 50%

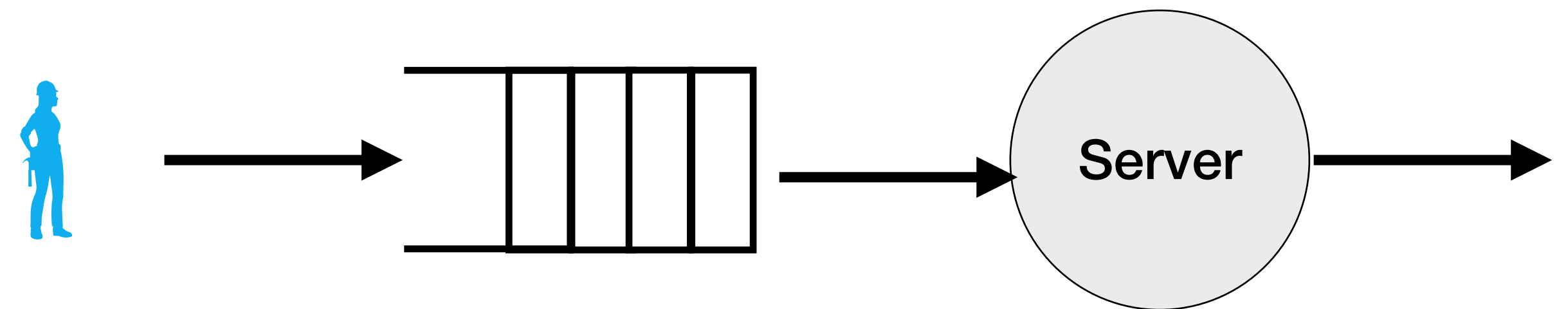
Schedule

- 1st session
 - Main
 - Trace
 - Test methods
 - Message
 - Event
 - Scheduler
- 2nd session
 - Main
 - Test Methods
 - Client
 - Queue
 - Server
- 3rd session
 - Main
 - Test methods
 - Engine
 - Gateway
 - Simulations

Simulations

M/M/1

- One Client
 - $\lambda = 4; 6; 8; 12$ per second
- One queue
- One Server
 - Service time = 8 per second
- Measure all the basic metrics
- Plot the results
 - With 95% confidence interval



M/M/1/4

- One Client
 - $\lambda = 4; 6; 8; 12$ per second
- One queue
- One Server
 - Service time = 8 per second
- Measure all the basic metrics
 - Including the number of dropped messages
- Plot the results
 - With 95% confidence interval

M/M/1/8

- One Client
 - $\lambda = 4; 6; 8; 12$ per second
- One queue
- One Server
 - Service time = 8 per second
- Measure all the basic metrics
 - Including the number of dropped messages
- Plot the results
 - With 95% confidence interval

M/M/3/8

- One Client
 - $\lambda = 4; 6; 8; 12$ per second
- One queue
- 3 Servers
 - Service time = 8 per second
- Measure all the basic metrics
- Plot the results
 - With 95% confidence interval

Performance Evaluation of Computer Systems

Class 09 - Simulator

Pedro Braconnot Velloso